Queen Mary
University of London

Introduction to Computer Vision

Coursework

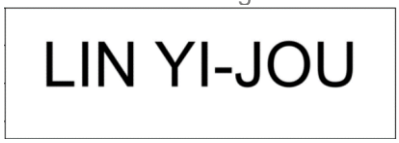
Submission 1

Your name Yi-Jou Lin

Student number 230362088

Transformations
Question 1(b):

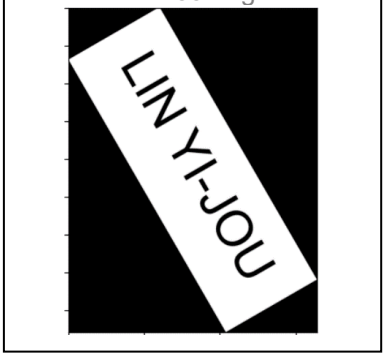
Your image



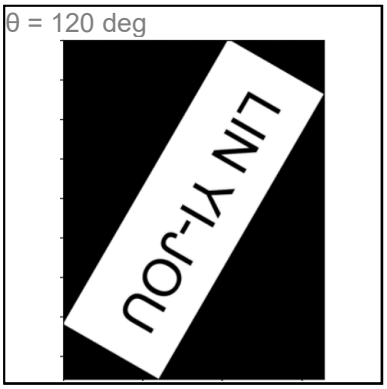
Rotated images:

θ = 30 deg

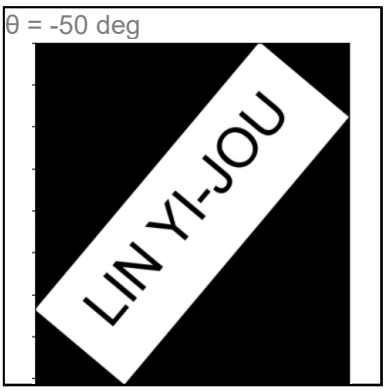
θ = 60 deg



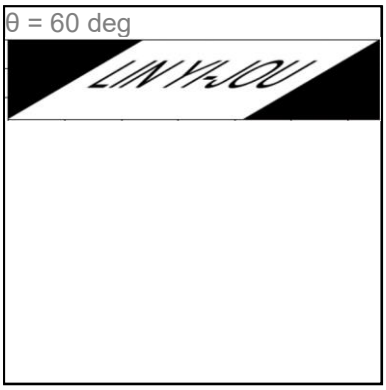
θ = 120 deg



θ = -50 deg



Skewed images:



Your comments:

Rotate Image

Given a rotation angle θ

$$R_\theta = \begin{bmatrix} cos\theta & -sin\theta \\ sin\theta & cos\theta \end{bmatrix}$$

Where the coordinate after rotation is

$$\begin{bmatrix} x_{new} \\ y_{new} \end{bmatrix} = \begin{bmatrix} cos\theta & -sin\theta \\ sin\theta & cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

However, the rotation coordinates and the original coordinates is not one to one. If we directly use rotation matrix to calculate new coordinates of current pixels and assign it to new images, some of the pixels on new image won't get any value. To solve this problem, I use the inverse mapping instead of using directly map the coordinate.

Inverse mapping

$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} cos\theta & -sin\theta \\ sin\theta & cos\theta \end{bmatrix}^{-1} \begin{bmatrix} x_{new} \\ y_{new} \end{bmatrix} = \begin{bmatrix} cos\theta & sin\theta \\ -sin\theta & cos\theta \end{bmatrix} \begin{bmatrix} x_{new} \\ y_{new} \end{bmatrix}$$

Rotation Centre

After the calculation of rotation, width and height will changed, therefore, the centre of the image should be used as an fixed point when assigning new pixel coordinates.

Rotation

For each coordinate (x,y) in new image
First compute the relative distance by (x-newcenter_x,y-newcenter_y)
Calculate according old pixels coordinate(x_rot,y_rot)
Adding the old center back by(x_rot+center_x,y_rot+center_y)

Skew Image

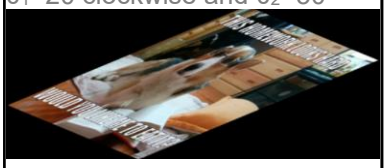
Given a skewing angle ω

$$\begin{bmatrix} x_{skew} \\ y_{skew} \end{bmatrix} = \begin{bmatrix} 1 & tan\omega \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

In the calculation of skewing, because only the width is changing, we only need centre of x-axis.

Question 1(c):

θ_1=20 clockwise and θ_2=50



θ_2=50 and θ_1=20 clockwise

Your comments:

$$\begin{bmatrix} x_{rot} \\ y_{rot} \end{bmatrix} = \begin{bmatrix} cos20 & -sin20 \\ sin20 & cos20 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}, \qquad \begin{bmatrix} x_{skew} \\ y_{skew} \end{bmatrix} = \begin{bmatrix} 1 & tan50 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Rotate20 -> Skew50

$$\begin{bmatrix} x_{new} \\ y_{new} \end{bmatrix} = \begin{bmatrix} 1 & tan50 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x_{rot} \\ y_{rot} \end{bmatrix} = \begin{bmatrix} 1 & tan50 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} cos20 & -sin20 \\ sin20 & cos20 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

$$= \begin{bmatrix} cos20 + cos20tan50 & -sin20 + cos20tan50 \\ sin20 & cos20 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Skew50 -> Rotate20

$$\begin{bmatrix} x_{new} \\ y_{new} \end{bmatrix} = \begin{bmatrix} cos20 & -sin20 \\ sin20 & cos20 \end{bmatrix} \begin{bmatrix} x_{skew} \\ y_{skew} \end{bmatrix} = \begin{bmatrix} cos20 & -sin20 \\ sin20 & cos20 \end{bmatrix} \begin{bmatrix} 1 & tan50 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

$$= \begin{bmatrix} cos20 & cos20 - sin20tan50 \\ cos20 & sin20tan50 + cos20 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

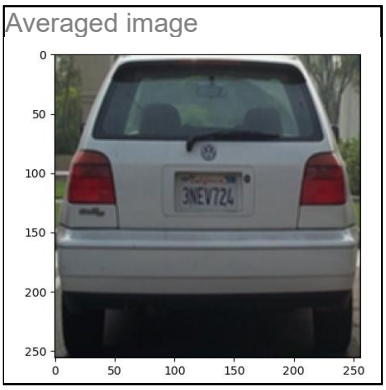
We can see from the above formula, where the matrices are different. Also, we can see from the figures shown, which is also different.

Convolution
Question 2(b):

Designed kernel:

$$k_{ave} = \frac{1}{3 \times 3}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Where k is the average kernel of 3×3 pixels



Your comments:

Given an $M \times N$ image, $k \times k$ kernel

Step of Convolution

1. img_zero = image after zero padding of adding $k//2$ rows/columns zeros of each border of image
2. apply convolution on image_zero with k_{ave}
3. output will be an $M \times N$ image

the convolution formula is as following

$$output[x][y] = \sum_{i=0}^{k-1}\sum_{j=0}^{k-1} img_zero[x][y] * kernel[i][j]$$

Averaged Image

After applying the average kernel, we can see that the edges of objects in the image is smoothed after convolution. Also, the background of the image is blurred by applying average kernel.

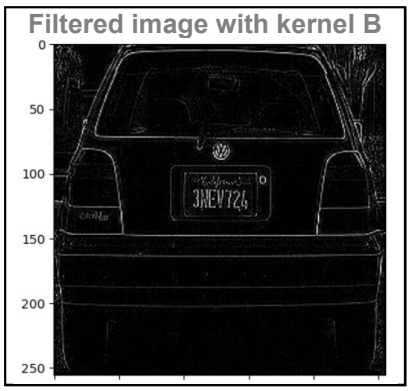
Zero padding

The reason I used zero padding before if we directly doing convolution to the image, the image we get after doing convolution will shrink to $(M-k) \times (N-k)$ image, which the border information will disappear after convolution.

Therefore, zero padding can

- save the border information.
- keep the image size after convolution.

Question 2(c):



Your comments:

$$A = \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}, B = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Kernel A

A is 3×3 Gaussian blur kernel.

Because the total sum of A is bigger than 1, when applying A with the image, during the calculation, I divide sum(A), so I can make sure the new value of the pixel is still in the range of [0,255].

As we can see on the image, after doing convolution with A, the image is smoother (blurry) than origin.

Also, the bigger size of gaussian kernel, the blurry an image will get after the convolution.

Kernel B

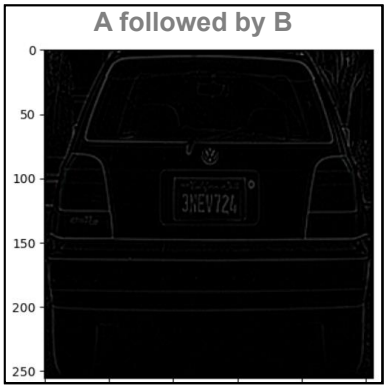
B is a 3×3 Laplacian kernel.

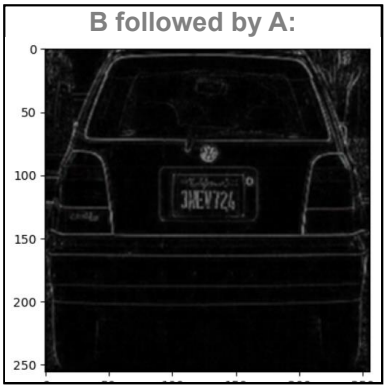
In the process of convolution, if the value is bigger than 255, the value will be set to 255, and if the value is negative, it will be set to 0.

By observing kernel B, we can see that if the original centre value is higher than the neighbour, the value after the convolution will become negative or close to zero, where it turns 0 after thresholding. On the other hand, the value will be threshold into 255 if the centre value is lower than the neighbour. Because the border most of the time is darker (small value), the value of the border may stay positive after the convolution.

Therefore, the Laplacian can extract the border after applying the kernel to the image.

Question 2(d):





Your comments:
A followed by A

Applying the same gaussian kernel twice can have a stronger blur effect than directly applying a bigger_size gaussian kernel, because it will blur the details in the image twice.

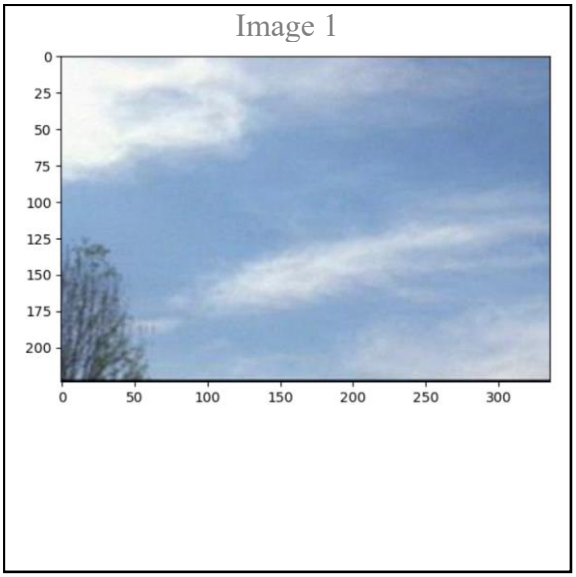
The figure above shown is the comparison of two methods. By observing the number of the car's license plate, we can discover the word is more blur of doing twice 3x3 gaussian.

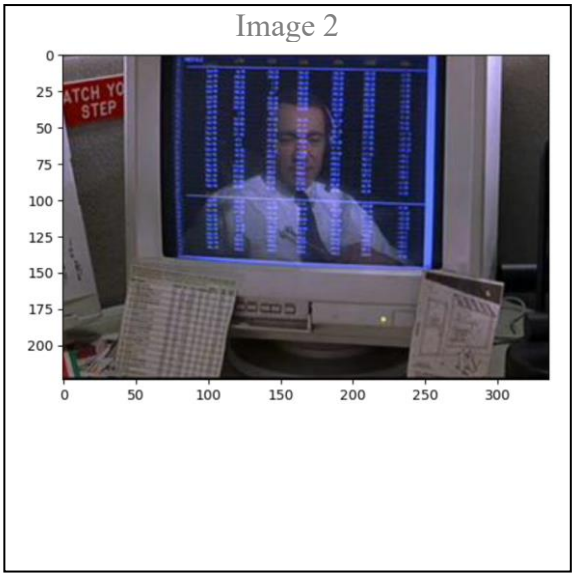
A followed by B
First applying kernel A, we can blur some unimportant detail first (denoise). After we denoise the image by A, B can sharpen the border. Therefore, by the result shown above, the border is clearer but at the same time less obvious.

B followed by A
By applying kernel B first, we can keep most of the border before blurring it. After doing convolution to A, since the border is only blurred by the kernel, most of the border will keep in the image. As the result shown above, lines of the border of the object is thicker, but at the same time there are also some noise in the image.

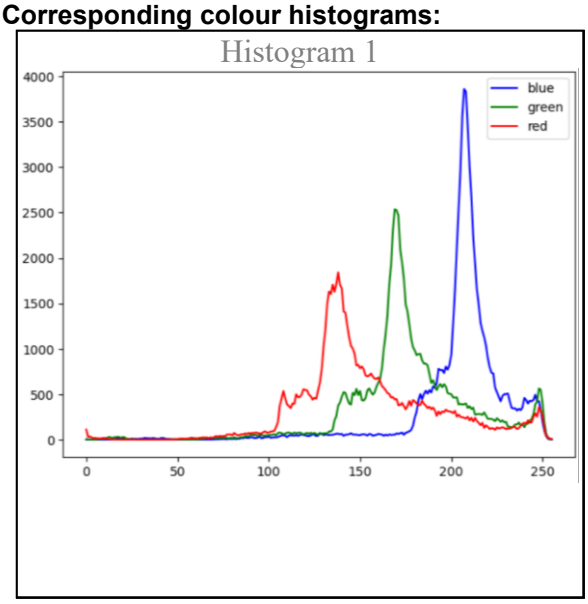
Histograms
Question 3(a):

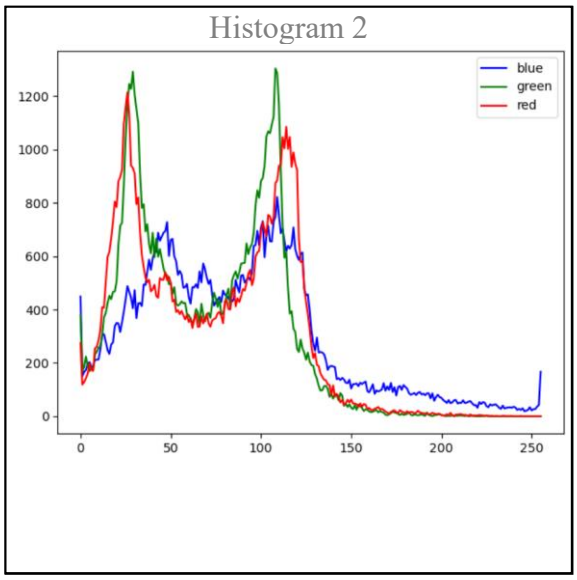
Two non-consecutive frames:





Corresponding colour histograms:





Your comments:

In this section, I use the for loop to count different frame's RGB histogram. The plot above shown the histogram of two non-consecutive frame and their histogram in different channel.

Histogram

From histogram, we can observe some information of the image.

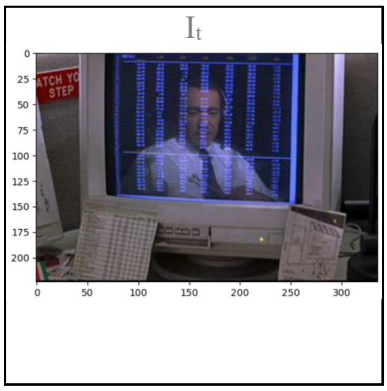
- Brightness of the image: for example, if most of the extreme value occurs on the right side (near 200, also called bright pixels), it means the image's brightness is higher.
- Image's colour or tone: if the image is taking under red light source (or similar to red light), or have lots of red object,…, the histogram tends to have high peak at higher value of R channel.

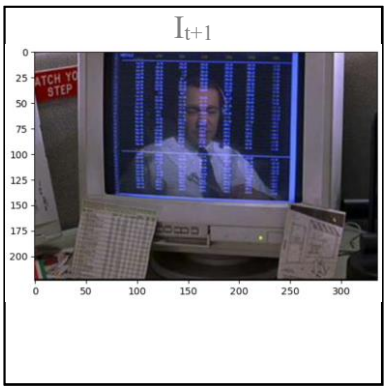
Checking histogram is correct

Except for observing the result, in this section, I also sum all the frequency value, and if the sum equals to image's pixels number (width*height) , the histogram is correct.

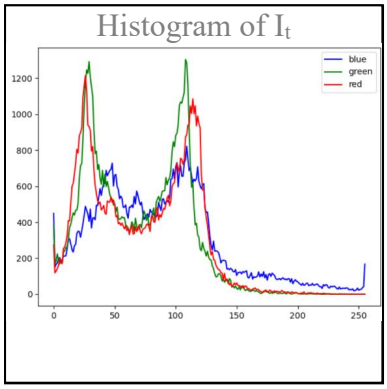
Question 3(b):

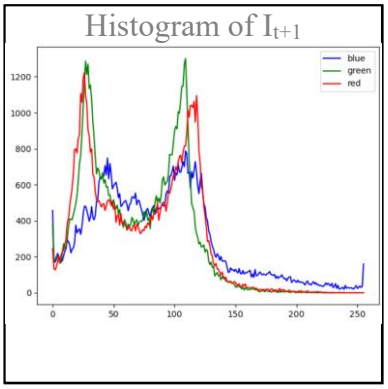
Two consecutive frames:

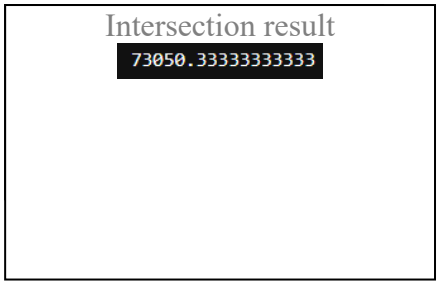




Histograms:

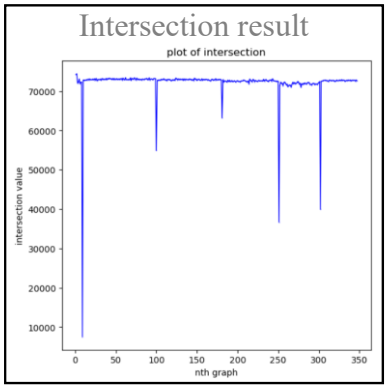


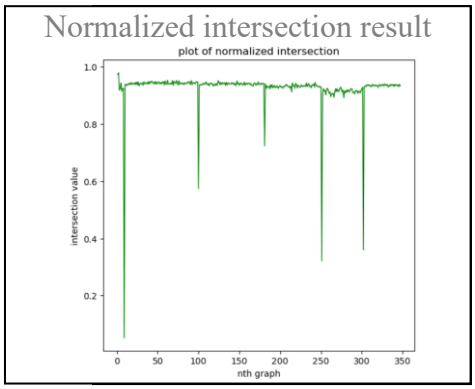




Question 3(b):

Intersection result for a video sequence:





Your Comments:

Given two histogram H1 and H2, both have R,G,B channel

Intersection

$$\boldsymbol{intersection\ value} = \frac{\sum_{channel=1}^{3}\sum_{i=0}^{255} \boldsymbol{min}(\mathbf{H1}_{channel,i}, \mathbf{H2}_{channel,i})}{\mathbf{3}}$$

Normalised Intersection

When normalising the intersection, two histogram should have intersection value = 1 if they are the same, also, if they are very different, their intersection value should be very small. Based on this concept, the normalising intersection value formula is design as follow :

For each channel,

$$\boldsymbol{normalised\ intersection\ value}_{channel} = \frac{\sum_{i=0}^{255} \boldsymbol{min}(\mathbf{H1}_{channel,i}, \mathbf{H2}_{channel,i})}{\sum_{i=0}^{255} \boldsymbol{max}(\mathbf{H1}_{channel,i}, \mathbf{H2}_{channel,i})}$$

For whole image:

$$\frac{\sum_{channel=1}^{3} \boldsymbol{normalised\ intersection\ value}_{channel}}{\mathbf{3}}$$

Compare

By observing the plot, the result is the same, however, after normalising, it can identify the similarity of different frame easier by the value between [0,1]

Question 3(c):
Comments:

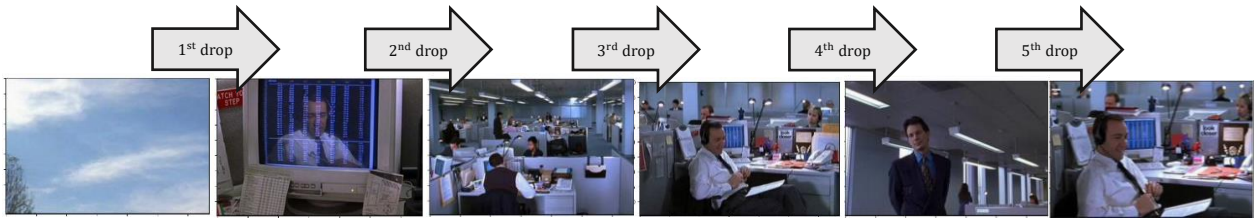
Intersection Value

The intersection value can show if the scene changes in a given video. Observing the intersection plot above, in some certain point, the value is dropping(the brightness changes or color tone changes). Therfore, the intersection of two histogram can tell if the scene is changes or not.

Changes in video and Intersection

In the video, if the object is moving but the scene is similar in general (local changes), the intersection won't have a drop in the plot, because the moving is not enough for changing the overall distribution of the histogram. To see this circumstance, we can see from the plot above that the intersection will have slightly fluctuation between two dropping.

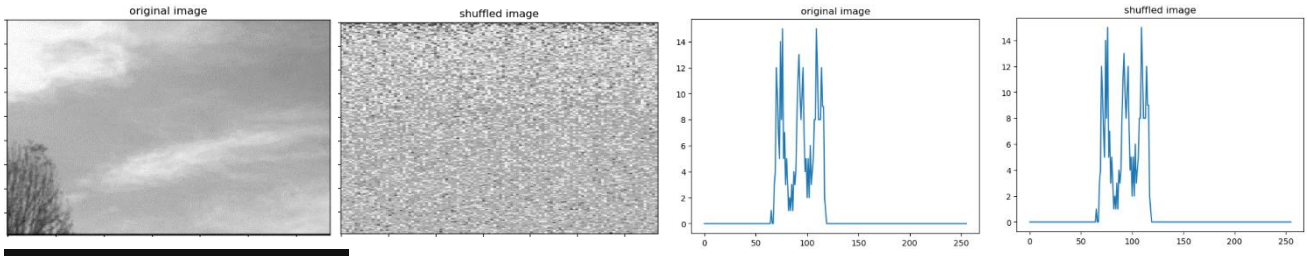
Drop points in the plot



The image showing above is the scene changes in the video, which also is the intersection drop on the plot.

- 1st *drop*: In the plot, the first drop is very small (near 0), if seeing by the first two graph, we can see that the scene's brightness changes very much, which will cause the intersection value became very small.
- 3rd *drop:* We can see in the changing scene of two photos of third drop, the background of the scene is very similar, this can explain the reason why the third drop is relatively smaller than others.
- 4th *drop and* 5th *drop:* we can see that the intersection value of the last two drop is similar, by directly observing the scene, we can discover that the fourth scene and the sixth scene above is also similar, and they all comare with the fifth scene, which can explain why the intersection value of fourth and fifth drop is close.

When will intersection failed of changing scene




```
intersection: 336.0
normalized intersection: 1.0
```

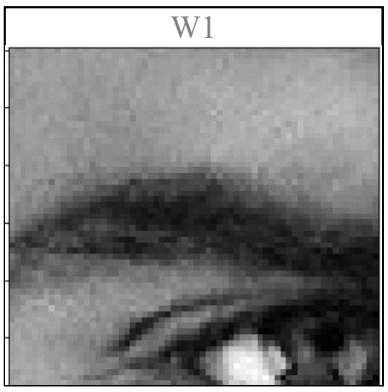
First shuffle the pixel value of one image, we can see the scene is different from original image to shuffled image, but the images have the same histogram.

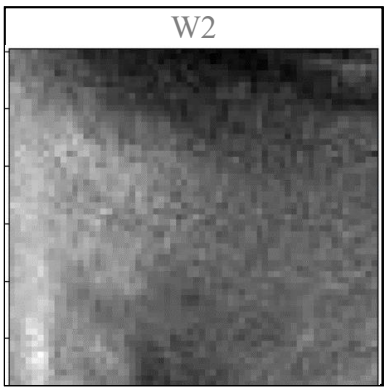
If we use both histogram to calculate the intersection, we can see that the normalized intersection=1, which supposed to represent the same image.

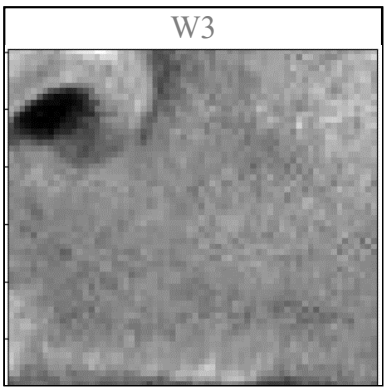
Therefore, the intersection failed of detecting changing scene when two different scene have same histogram.

Texture Descriptors and Classification
Question 4(a)

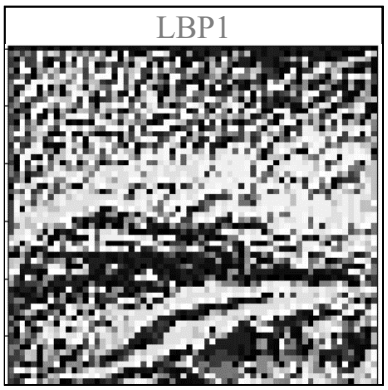
Three non-consecutive windows

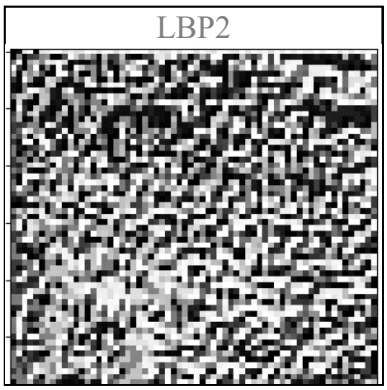


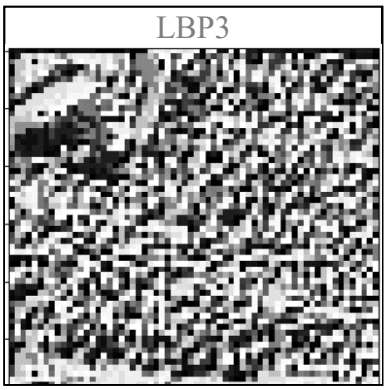




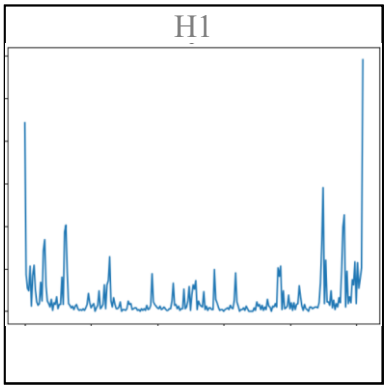
LBP of windows

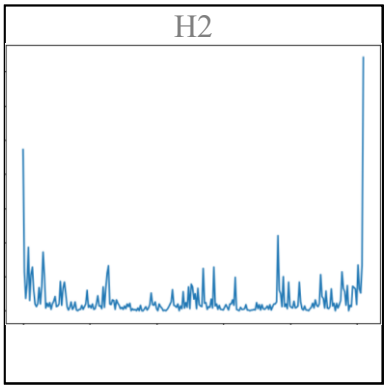


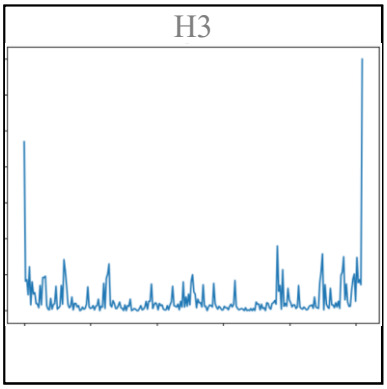




Histograms of LBPs



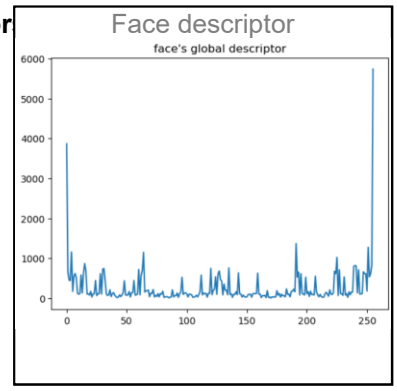


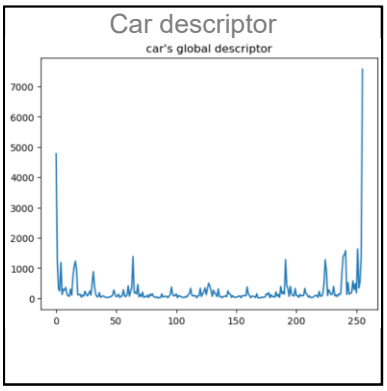


Question 4(b)

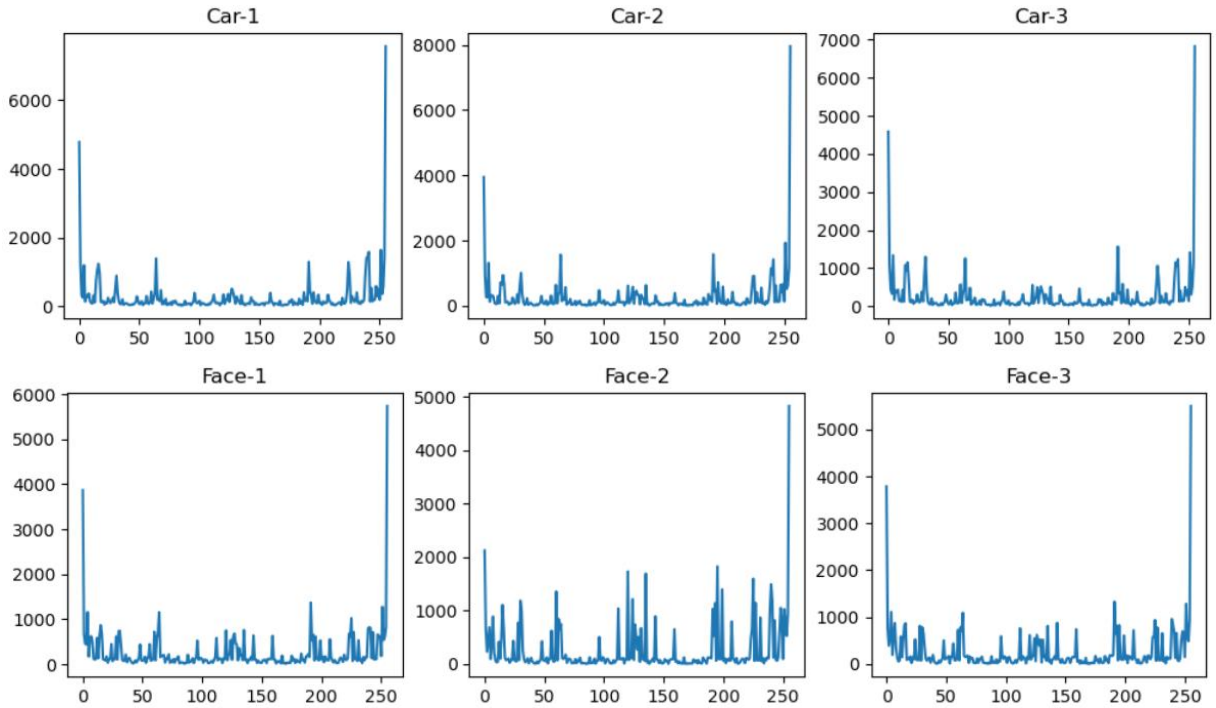
Two example images:

Descriptor





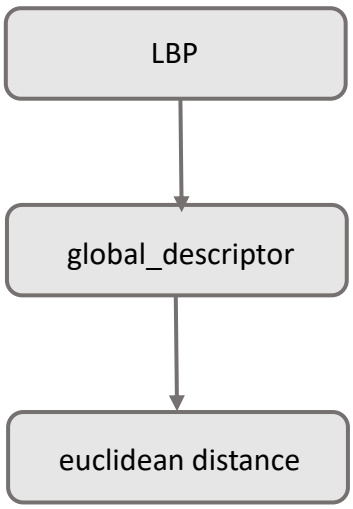
Your comments:



By looking at the descriptor above, it is hard to directly use the histogram to classify two class, because there is no significant value or some continuous value that can used to classify two classes. However, there is still some slightly difference of every value's frequency, if sum all of them together, the value could be enough for classifying.

Question 4(b)

Block diagram of classification process



Input: image, window_size
- Test if window_size is factor of both width and height
- Divide the image by window_size
- Calculate each window's LBP

Return: all window's LBP, histogram of each window's LBP

Input: h (histogram list of N window's histogram)
- Sum row's value of h^T

Return: global descriptor (result of sum)

Input: two global descriptors, h1 and h2
- Calculate the Euclidean distance of two descriptors.

Return: distance value

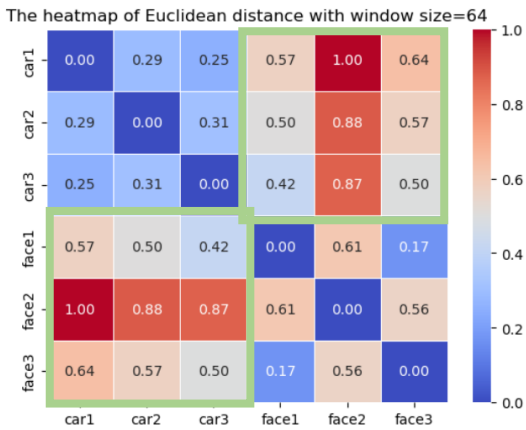
The Euclidean distance's formula is as follows,

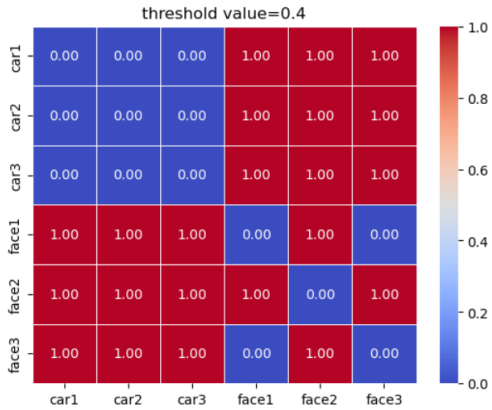
$$\boldsymbol{d} = \sqrt{\sum_{i=0}^{255} (\boldsymbol{h1}_i - \boldsymbol{h2}_i)^2}$$

The lower the value is, the similar the images are.

Your comments:

After having all image's global descriptor, we can use a heatmap to see the similarities between descriptors



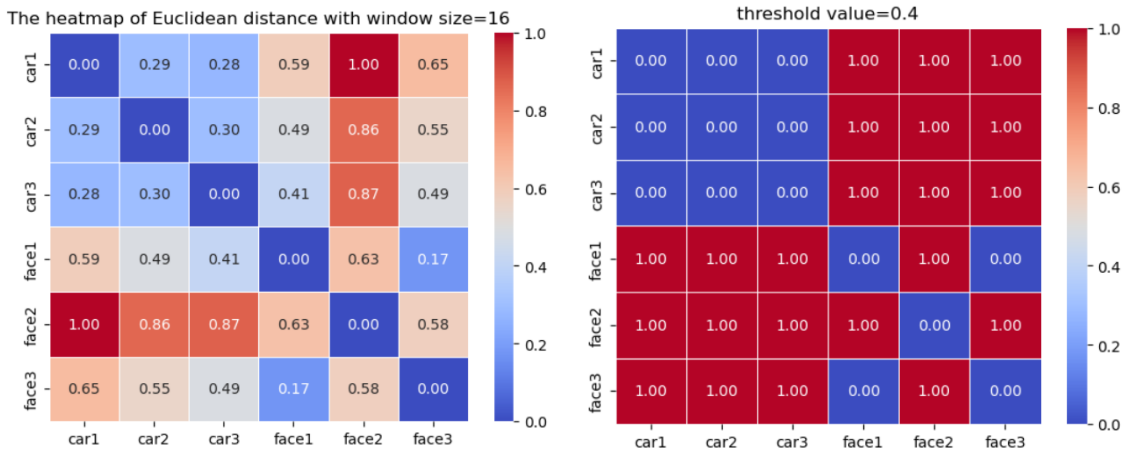


If the classifier is good enough, the green region of the heatmap should tend to be red.

To see the difference more clearly, threshold the result by 0.4, we can see that car and face can be well classified by the classifier. Also, car's value in the classifier is really low, which can identify as same class. However, face's classification if nod good enough, where they are easy classified as different class.

Question 4(c)

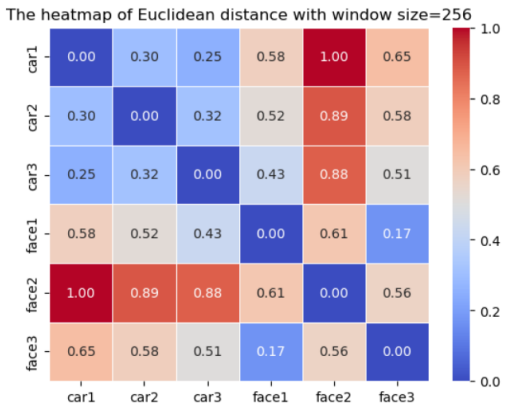
Your comments:

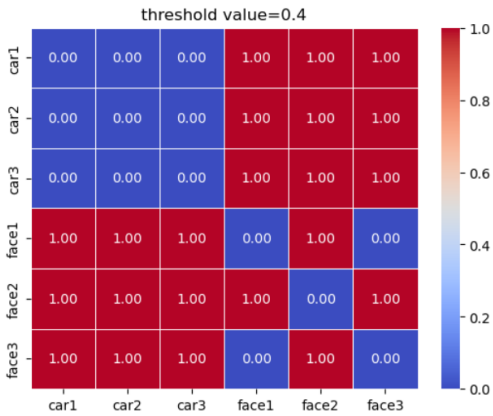


Decrease the window size to 16, and threshold the value by 0.4, the result keeps the same as previous one. And the similarity of face1 and face2 is 0.2 more than previous one.

Question 4(d)

Your comments:





After increasing the window size, the result is still the same, there is only a slightly different with previous two. To sum up, the window size of LBP doesn't have much impact on the classifier.

Question 4(e)

Your comments:

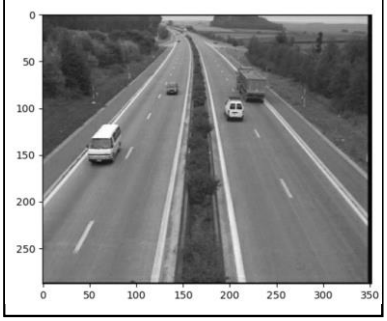
For dynamic textures of the video, since the object in a video won't have large difference when moving, each most of the objects in the same class can be classified into same class easier than datasetA. However, if different texture's performance in global descriptor is very similar, the classification might be wrong at the same time. Also, because the video has one more factor, time, when doing LBP, temporal information can also be included in descriptor.

Object Segmentation and Counting
Question 5(a)

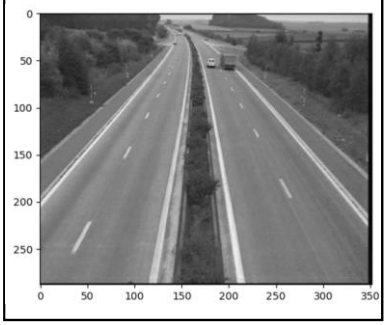
Original frames:

Reference frame

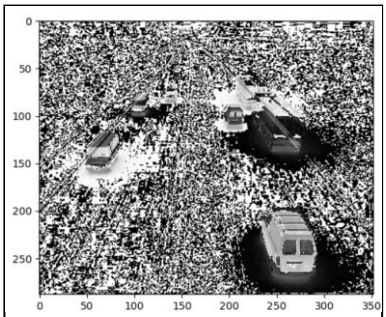
Selected frame 1
20th

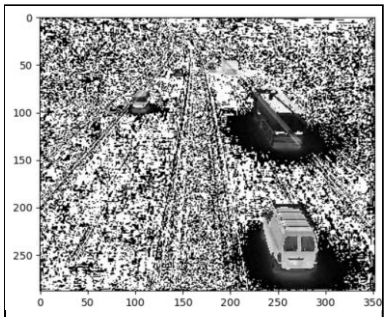


Selected frame 2
60th

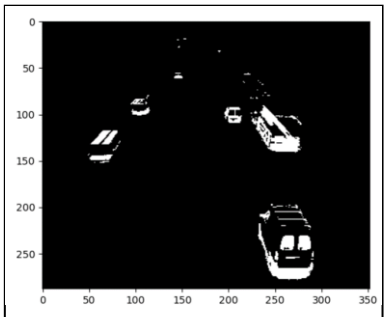


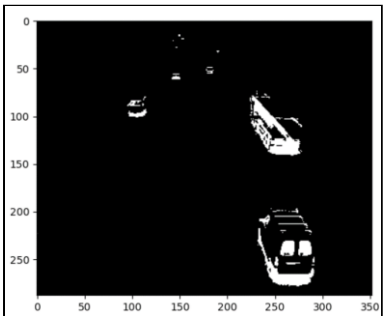
Frame differencing:





Threshold results:

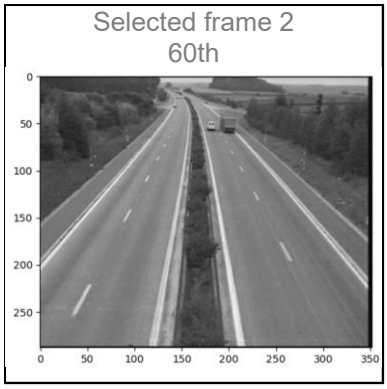




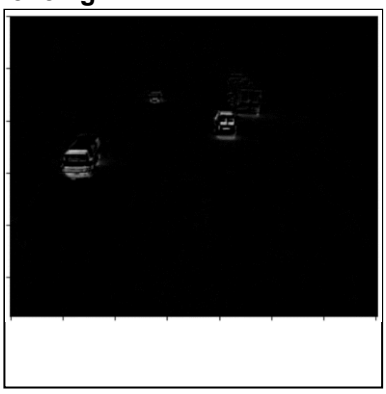
<Threshold value=40>

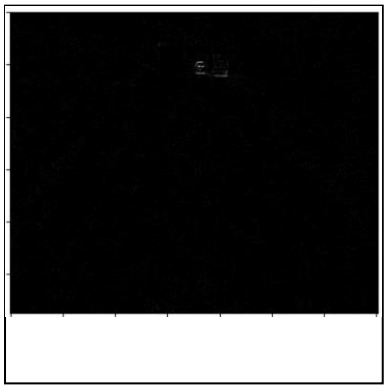
Question 5(b)

Original frame:

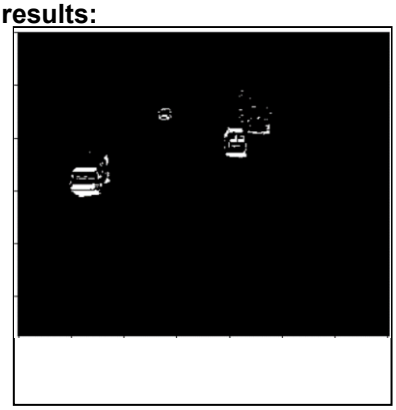


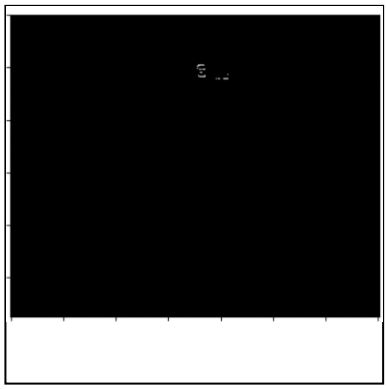
Frame differencing:





Threshold results:





Threshold value = 30

Your comments for 5a,5b:

Frame Differencing

Given $frame_{ref}$ and $frame$, where frame's size = (h,w).

$$diff[x][y] = frame_{current}[x][y] - frame_{ref}[x][y], \qquad \forall\, x, y \text{ where } x \in [0, height) \text{and } y \in [0, width)$$

After doing differencing, threshold the value.

$$\begin{gathered} if\ diff[x][y] \leq threshold: \\ diff[x][y] = 0 \\ if\ diff[x][y] > threshold: \\ diff[x][y] = 255 \end{gathered}$$

Reference Frame is First Frame

When using the first frame as the reference frame, because there are some objects with dark pixels, for example, the windows of the van, the truck (darker than the road), which means if using the first frame as the reference frame, during the calculation (frame-frame_ref), the new pixel value of those area is easily being higher than the threshold unless it is subduct by other darker pixels. Therefore, the differencing result will keep the dark pixels.

first frame frame

In the above example, we can see the larger threshold value we take, we can threshold the dark pixels to 0. However, when the threshold is able to eliminate dark pixels' region, other features of current frame might be threshold to 0 at the same time. Therefore, the first frame is not an ideal reference frame.

Reference Frame is Previous Frame

During calculating the differencing of previous frame and current frame, most of the object can keep the shape. However, if the car is too far away from the camera, the moving distance between the reference frame and the current frame may not be big enough to been keep if the threshold value is too big.

previous frame current frame

In the graph shown above, we can see that if the threshold value is small, we can easily keep the shape of the car, however, we will keep the noise at the same time. The threshold value around 10's performance is the best. To sum up, using previous frame as the reference frame keep most of the shape of the image, but has problem with keeping the object far away from the camera.

Question 5(c)

Generated background

Your comments:

Weighted Temporal Averaging Algorithm

This algorithm updating background by using the below formula,

$$background_{new} = \alpha \cdot frame_{current} + (1 - \alpha) \cdot background$$

Where α is a constant.

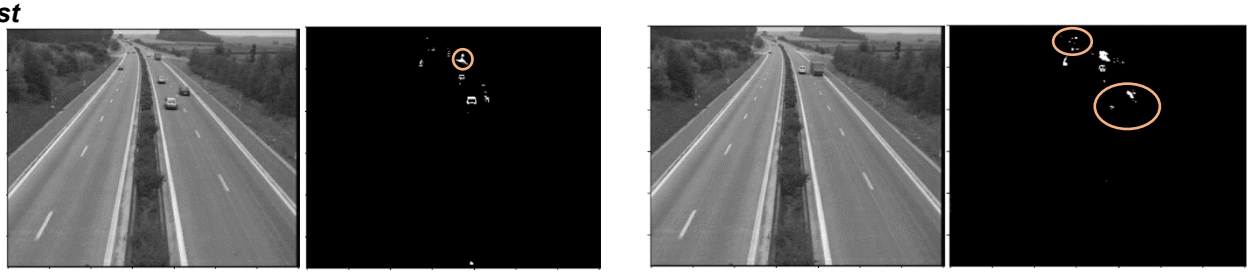
Best Alpha

$\alpha = 0.01$ $\alpha = 0.05$ $\alpha = 0.1$ $\alpha = 0.5$

In the above graph shown, aside from the left and the right figure, the rest are the result after applying different alpha. By observing the formula above, we can discover that, the lower alpha we picked, the result will tend to be more like the first frame. And the bigger is the alpha, the algorithm will tend to generate the background more like the last frame.

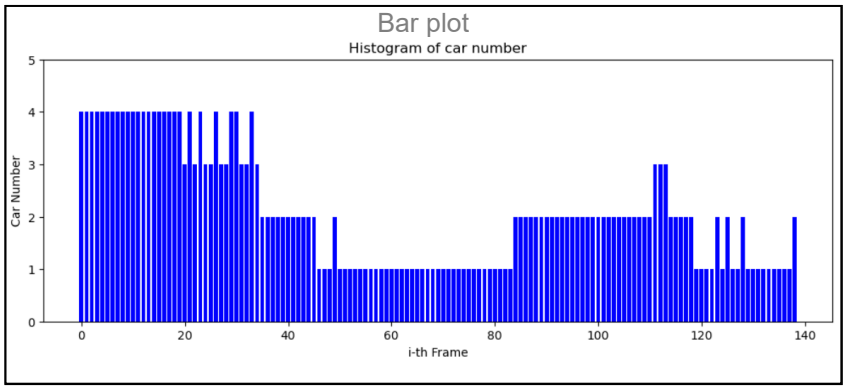
With all the analysis in last part, the ideal reference frame is without any cars, objects on the highway. By looking at the result provide above, it is good to set alpha=0.05, which is the cleanest background.

Test



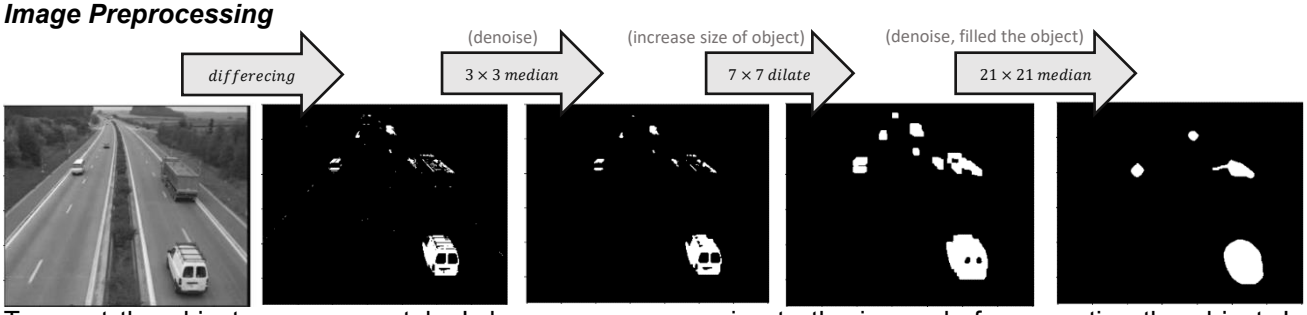
Above is the result of using background as the reference frame, we can see that if even if the car is small, after differencing, they can keep the small car which is far. However, the residual shadow in the back of the highway of the background can cause extra noise when differencing the frame (orange circles).

Question 5(d)



Your comments:

Image Preprocessing



To count the object more accurately, I do some preprocessing to the image before counting the object. In preprocessing, the ideal image should turn all the objects in the image to a filled white shape.
Firstly, denoise the differencing image by 3x3 median filter. By the result, we can discover that the result still has some noise but is only slightly smaller of some background object. To avoid remove the object with noise together is the 21x21 median. The dilate filter can help the situation.
In dilate, we can change the pixel to 255 if there is connected pixel around it is 255. By this concept, the white area can dilate to bigger size.
In the end, using 21x21 median filter to remove rest of the noise, and turn the inside black region to white region at the same time.

Counting Object

In this function, labeled the connected components by using DFS to determine if neighbors of current pixel are in the same connected components or not. The maximum label number is the number of objects.

Advantages

In the bar plot shown above, we can found that the fluctuation of number of cars in the video is matched with the video. Also, this method is able to count the cars behind.

Disadvantages

However, the differencing image still have some noise on the back of the highway, and it is too hard to separate noise and small objects (both shape and size are similar). Also, when dilating the object, small objects and noise in the image is easily combinig together if want to dilate the car's window at the same time. To sum up, there is a lot of reason why the result is not very accurate.